Meet-up talk outline (v2, simple English) — "Single Responsibility Principle in AI: one door to the LLM"

Version 2 of `outline.md`, rewritten in simple (B1) English. This file is edited by hand — it is not generated from `deck.html`.

Planned length: **44 min** · 23 slides · English.

Deck controls: → / Space / PageDown = next (shows builds first) · ← = back · P = pause · R = restart animation · – / + = animation speed (0.25×–3×, saved) · 0 = 1× · F = full screen · S = window with speaker notes.

Opening — 2.5 min

1. About me *(0.5 min, starts at 0:00)*

- Say who I am and what I do at Spin.AI. Say why AI in many services is a real problem for us. Keep it under 30 seconds.

2. Single Responsibility Principle in AI *(0.5 min, starts at 0:30)*

- Today I tell a story about architecture: AI inside a big group of microservices. And about a 20-year-old principle that is very important again today.
- This talk is about architecture, not about a framework. There are no annotations, no versions and no class names on the slides. You can relax.
- Typed Goose is the study project for this talk. It is a small system. We built all of it, mostly together with an AI assistant. I will say more about this at the end.

> 😄 If you came for stories about framework migrations — we can talk about that at the after-party.

3. One reason to change *(1.5 min, starts at 1:00)*

- Optional quick question to the room (say it, it is not on the slide): "Who has an LLM API key in the config of more than one service?" Count the hands. Then: "Keep your hand up if you know exactly how many." Usually the hands go down. "…I thought so." This one moment is the whole talk.
- SRP means: a module has one reason to change. Billing changes because tax rules change — that is OK, it is its job. Billing changes because an LLM company renamed a model — that is a second reason to change.
- Take this second reason and multiply it by every service that talks to an LLM. This is the problem we look at today.

> 😄 Uncle Bob gave us SRP in the early 2000s. Twenty years later we gave every service a second reason to change: an API key.

Part 1 · The problem — 11.5 min

4. Meet the fleet *(2.5 min, starts at 2:30)* 🎞 **looping animation**

- A normal company has 20–30 services. Different teams own them, and each team releases at a different time. Each service has one job. Billing does billing. Search does search. Support… supports.
- This is the clean picture. Then the boxes start to shake: these are the hidden connections. Everybody knows about them, but nobody draws them.
- And now the product team comes in with the big question of the decade: "Can we add AI?"

> 😄 Our architecture diagram — the version for management. …And the real version.

5. The first one *(3 min, starts at 5:00)* **looping animation**

- The Support team wants short summaries of long tickets. One team, one service, one line to an LLM provider.
- They put the key in an environment variable, write the model name directly in the code, and add a small retry loop. The bill goes to… somebody's credit card.
- And to be honest: for ONE service this is the right decision. Do not build a platform for only one user.
- But look at the red robot: Support now has two jobs. One job is to help customers. The other job is to be an LLM client.

> 😄 It took one sprint. The demo was great. The PM cried. Happy tears — for now.

6. The second, the third, the fifth… *(3 min, starts at 8:00)* **looping animation**

- Success spreads fast. Search wants semantic search. Reports wants summaries in text. Marketing wants texts for ads. Docs wants questions and answers.
- Everybody copies Support's code — or they don't, and they write their own. Marketing read a blog post, so they use a different provider. Of course.
- Now there are five keys in five secret stores, three SDK versions, two or three providers. And nobody knows how big the monthly bill is.
- Story moment (animation): Reports runs a job every night, and it sends a lot of requests with the shared key. The provider answers "429 – too many requests". And who gets the error? The Support summary that customers see. At 2 a.m.

> 😄 Microservices: because one big ball of mud was not distributed enough.

7. Dozens *(3 min, starts at 11:00)* **looping animation**

- One year later. Twenty services talk to LLMs, and some of them talk to two providers. Count the keys: 23.
- To change a key, you need 23 pull requests in 15 teams. When a provider stops supporting a model, the whole company has an emergency.
- Finance asks: how much do we spend on AI, for each team? Nobody knows. Compliance asks: do we send personal customer data to an LLM? You must check 23 code bases.
- Every team chose its own timeouts and retries. Some services retry forever. On an API that costs money.
- This breaks SRP at the level of the whole company: the "LLM client" job is copied into every service. So every change at the provider becomes a change in every service.
- Only five services did not change: Auth, Shipping, Invoices, HR, Audit — the only teams that read the security policy.

> 😄 When a provider stops supporting a model, you learn which teams read release notes. Spoiler: no team.

Part 2 · One door — 6.5 min

8. One door: the AI Gateway *(2.5 min, starts at 14:00)* **looping animation**

- The solution is a very old trick: take the job that is copied everywhere and give it ONE owner.
- An AI Gateway is a service with one job: to talk to LLMs. All other services go back to their own job.

- Look at the small labels: keys, provider abstraction, model tiers, timeouts, rate limits, cost, observability, sync and batch. Many features — but one responsibility: to be a good LLM client.
- Services ask in business words ("summarize this, fast tier"), not in provider words ("model X, temperature 0.2, header Y").

> 😄 We did not remove the complexity. We gave it a home address.

9. Pros & cons — honestly *(2 min, starts at 16:30)*

- Single point of failure: yes. So you run more than one copy of it, like any other important service.
- One extra network hop: a few milliseconds before a call that takes seconds. Nobody will notice.
- The "god gateway" risk is again about SRP. Imagine someone adds "if the customer is ACME, use the special prompt" into the gateway. Then the SRP problem did not go away — it just moved to a new place. Business prompts belong in the business service.

> 😄 We changed 23 problems into one. It is a very big one — but at least it is ours.

10. A solution — not the solution *(2 min, starts at 18:30)*

- Say it clearly: this is not the only right way. It is the way we built, to learn what it needs.
- A shared library: no extra hop. But every change means N upgrades, and you need the library in every language you use.
- Ready-made proxies are a completely good choice: "buy before you build".
- Go to the next part: in our study project we built one. It has the two kinds of traffic you will always see — a person who waits (chat) and a machine that waits (file processing).

> 😄 Architecture answer #1: "it depends". Architecture answer #2: "it still depends — but now there is a diagram".

Part 3 · Sync: chat — 5.5 min

11. Sync flow ① — the question goes in *(2 min, starts at 20:30)* 🎞️ **looping animation**

- We show only the business parts: UI, Chat Service, AI Gateway, LLM provider. Each box has exactly one job — it is written under the name.
- The UI shows things on the screen. The Chat Service owns the conversation: it adds the history, and it makes sure that each chat has only one answer in progress at a time.
- The AI Gateway owns the LLM call: it chooses the provider and the model tier. The Chat Service never knows the provider's model name.
- The speed control (– / +) is useful here if people want to see the steps slowly.

> 😄 Look at the question. We only test with real-life use cases.

12. Sync flow ② — the answer streams back *(2 min, starts at 22:30)* 🎞️ **looping animation**

- The LLM writes the answer one piece (token) at a time. No step waits for the full answer — each piece goes on as soon as it arrives.
- People feel speed by the time to the first word, not by the total time. With streaming, 8 seconds of silence become 8 seconds of reading.
- The Chat Service sends the pieces on AND collects the full answer. At the end it saves the conversation history.
- Real story: once our streaming worked perfectly through all four steps — but the UI did not scroll. So the answer looked like it came all at once. We measured every step before we found the bug: it was

the scroll bar.

> 😄 Streaming: a loading spinner that really tells you something.

13. Sync flow — the whole round-trip *(1.5 min, starts at 24:30)* **looping animation**

- Both halves together. Every step is a stream; nobody waits for the full answer.
- SRP check: change the LLM provider — only the AI Gateway changes. Change the rules for chat history — only the Chat Service changes. Change the UI design — only the UI changes.

> 😄 Four boxes, four jobs, zero API keys in the Chat Service. Uncle Bob would be proud. Or at least less disappointed.

Part 4 · Async: files — 7 min

14. Async flow ① — submit & wait *(2.5 min, starts at 26:00)* **looping animation**

- The second kind of traffic: nobody is looking at a spinner. The task is to summarize many uploaded files.
- The Calculation Service owns jobs and files. It saves the job as PENDING, sends back a job id at once, and puts one message for each file on the message broker.
- The UI asks for the status again and again (polling). Polling is boring — and that is good: it has no state, it works with caches, and it is good enough for jobs that take minutes.
- Note: the Calculation Service does not know that an LLM exists. It only put work in a queue.

> 😄 The UI polls like a child in the back seat of a car: are we there yet? Are we there yet?

15. Async flow ② — batch processing *(3 min, starts at 28:30)* **looping animation**

- The AI Gateway takes messages from the broker and collects them into a batch — "5 messages or a few minutes, whatever comes first".
- It sends ONE batch to the provider's Batch API. A batch costs about half the price of live calls. The cost is time: minutes, sometimes hours.
- Then it waits and checks the batch status from time to time. It remembers the open batches, so a restart does not lose them.
- The results go back to the broker, the Calculation Service saves them, and the next UI poll finally says DONE.

> 😄 Batch API: 50% cheaper, delivered some time between now and your next stand-up.

16. Async flow — the whole round-trip *(1.5 min, starts at 31:30)* **looping animation**

- Everything together, with a fast clock: in real life the pause in the middle takes minutes. This pause IS the point — this is where we save 50%.
- SRP again: the UI shows and asks; the Calculation Service owns jobs; the database remembers; the broker delivers; the AI Gateway talks to the LLM. If you move to another provider's batch API, you change only one service.

> 😄 Async is just sync without trust issues.

Part 5 · The whole system — 6 min

17. The whole system — platform pieces *(2.5 min, starts at 33:00)* **looping animation**

- Now the real picture. Nginx is the front door: it gives the UI to the browser and sends API calls further. The API Gateway sends requests to services by their name.
- Eureka is the phone book: services register when they start. The API Gateway and the Chat Service ask it: "Where is chat-service?" / "Where is the AI gateway?".
- Postgres is "the database", Kafka is "the message broker". Calc, Kafka and Postgres do nothing on this slide — this animation shows the chat flow.
- Yes: API Gateway and AI Gateway. Only one letter is different, but the jobs are completely different. One sends requests to services, the other talks to LLMs.

> 😄 Eureka: the only service that has the name of the feeling you get when it finally works.

18. Watching it — logs, traces, health *(2 min, starts at 35:30)* **looping animation**

- The business traffic is grey on purpose. This is the layer that watches the system, and each part here also has one job.
- Logs: every service writes structured JSON logs → Logstash → Elasticsearch. In Kibana, people search the logs.
- Traces: every step sends a span to Jaeger, and Jaeger puts them together into one trace. The first time you open it, you see that the LLM takes about 95% of the time.
- Spring Boot Admin asks Eureka which services exist. Then it checks the health of each service — one dashboard with (hopefully) green lights.

> 😄 Logs tell you what happened. Traces tell you where. Nobody tells you why.

19. Everything at once *(1.5 min, starts at 37:30)* **looping animation**

- Chat, files and monitoring all run at the same time. About fifteen containers, and everything starts on one laptop with one command.
- Optional: go to the live demo here — send a chat message, upload two files, open the trace in Jaeger.

> 😄 Also running, but not on the diagram: my laptop fan — the loudest service in the architecture.

Built with AI — 3 min

20. How it was built: an AI pair, with a diary *(1.5 min, starts at 39:00)*

- We built the project with an AI coding assistant as a pair programmer — not as a vending machine where you put in prompts.
- The main problem: every session starts with an empty memory. So we write down everything important, in the repo, in files that the AI reads first.
- Project instructions = the house rules. Steps = the plan, cut into parts so that one session finishes one step. ADRs = why we made decisions. Changelog = what really happened.

21. The cross-session loop *(1.5 min, starts at 40:30)* **looping animation**

- One loop for each session: read the next step → brainstorm with the human → plan → build → check → update the step file and the changelog → stop.
- "Check" means real checks: the build is green, the containers are up, AND a real request goes through the feature. Look at how the packet jumps back once at "Verify" — that happened a lot.

- On the side: when something good for the demo happened, we saved a short note. At the end of every session, the full raw transcript was saved automatically. This talk is really made from those notes.
- SRP for the AI: one session, one step, one responsibility.

> 😄 Stopping is a feature. The hardest prompt to write is "that's enough for today".

Wrap-up — 2 min

22. Takeaways *(1.5 min, starts at 42:00)*

- Four takeaways, one sentence for each. Make a pause after the first one — it is the title of the talk.

23. Thank you *(0.5 min, starts at 43:30)*

- Say thank you to the organizers. Answer questions. If the room is quiet, offer the live demo.